

Problems with RNN (Recurrent Neural Networks)

RNNs are powerful for **sequential data** (text, audio, time series), but they have **serious weaknesses** when trained over long sequences.

1. How RNNs Learn

At each time step:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

- x_t : input
- h_t : hidden state (contains info from previous steps)
- y_t : output
- W_{xh}, W_{hh}, W_{hy} : shared weights

Training uses **Backpropagation Through Time (BPTT)** to update weights based on **errors from all previous steps**.

2. Main Problem — Vanishing & Exploding Gradients

When we backpropagate through many time steps, gradients get multiplied repeatedly by the derivatives of activations (like tanh or sigmoid).

► If derivative < 1 → Vanishing Gradients

- Gradients shrink exponentially
- Early time steps get almost **no learning signal**
- The network **forgets long-term context**
- The model only learns **short-term dependencies**

Example:

While predicting the last word in “The cat that chased the mouse was *hungry*,” RNN may forget “cat” after many steps.

► If derivative > 1 → Exploding Gradients

- Gradients grow exponentially
 - Training becomes **unstable**
 - Weights take **huge values**
 - Model may diverge (loss becomes NaN)
-

Why This Happens (Intuition)

Each hidden state gradient depends on a chain of multiplications:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k}$$

That product either:

- **shrinks toward zero** (if < 1) → vanishing
- **blows up** (if > 1) → exploding

The longer the sequence, the worse the problem.

3. Long-Term Dependency Problem

This is the **biggest limitation** of vanilla RNNs.

RNNs theoretically can remember long-term information, but in practice because of vanishing gradients they **cannot connect distant events**.

► Example

Let's say we want to predict:

"The clouds are dark, it's going to ____."

To predict "rain," the model needs to **remember "clouds"** from the beginning.

But by the time it reaches the last word, that information has **faded away**.

Short-term context (recent words) is learned easily.

But **long-term context** (earlier words) is lost due to gradient decay.

4. Computational Problems

1. Sequential Processing

- Each step depends on the previous one.
- You **can't parallelize** the training easily.
- So RNNs are **slow** compared to CNNs or Transformers.

2. Difficult Training

- Due to unstable gradients, training needs **careful tuning** of learning rate and initialization.

3. Memory and Time Expensive

- Long sequences → more time steps → more computation → slower convergence.
-

5. Solutions / Improvements

(a) Gradient Clipping

Limit the gradient size to prevent explosion.

```
optimizer = tf.keras.optimizers.Adam(clipvalue=1.0)
```

or manually:

```
grads = [tf.clip_by_value(g, -1.0, 1.0) for g in grads]
```

(b) Better Architectures — LSTM & GRU

LSTM (Long Short-Term Memory)

and **GRU (Gated Recurrent Unit)** were designed to fix **vanishing gradients**.

They use **gates** that:

- Control **how much info to keep or forget**
- Allow **gradients to flow more easily through time**
- Capture **long-term dependencies**

So, instead of passing everything blindly, they **“remember” important info** and **“forget” unnecessary parts**.

(c) Proper Weight Initialization

- Use methods like **Xavier** or **He initialization** to avoid extreme gradient values.
-

(d) Use ReLU or Leaky ReLU

- These activations have derivatives closer to 1 → less vanishing.
-

(e) Truncated BPTT

- Don't backpropagate through the entire sequence.
 - Limit it to, say, 10 or 20 time steps — reduces vanishing/exploding effect and speeds up training.
-

6. Visual Summary

Problem	Cause	Effect	Solution
Vanishing gradient	Derivatives < 1 during backprop	Network forgets early info	LSTM/GRU, ReLU, proper init
Exploding gradient	Derivatives > 1	Training unstable	Gradient clipping
Long-term dependency	Vanishing gradient	Can't learn far relationships	Gated RNNs (LSTM/GRU)

Slow training	Sequential nature	Not parallelizable	Use Transformers later
Computation cost	Many steps, shared weights	High memory/time usage	Truncated BPTT

7. Simple Analogy

Imagine trying to remember a story word by word:

- After 2–3 sentences, you remember the start.
- After 20 sentences, you forget the beginning.

That's **vanishing memory** — exactly what happens in RNNs.

LSTMs and GRUs act like a **notebook** — they store key points to recall later.

➤ Final Takeaways

- RNNs process sequential data but struggle with **long-term memory**.
 - The main training issues are **vanishing** and **exploding gradients**.
 - These make it hard to learn **long-range dependencies**.
 - Solutions include **LSTM**, **GRU**, **gradient clipping**, **better initialization**, and **truncated BPTT**.
-